

Actor-Critic

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

Improving the policy gradient with values

Recap: policy gradients

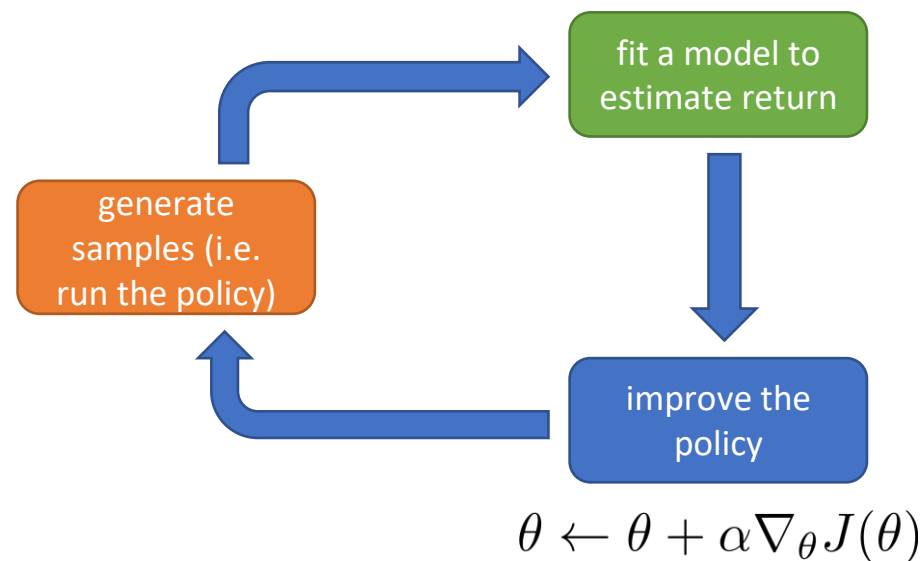
REINFORCE algorithm:

- 
1. sample $\{\tau^{(i)}\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
 2. $\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)}) \underbrace{\left(\sum_{t'} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right)}_{\hat{Q}_t^{(i)}}$
 3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)}) \underbrace{\hat{Q}_t^{(i)}}_{\text{"reward to go"}}$$

“reward to go”

$$\hat{Q}_t^{(i)} = \sum_{t'=t}^T r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})$$



Q也就是从t时刻开始到轨迹结束的累积奖励。这种形式和之前的公式本质一致，只是将累积奖励单独提取出来，更便于理解。

Improving the policy gradient

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{Q}_t^{(i)}$$

$$\hat{Q}_t^{(i)} = \sum_{t'=t}^T r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})$$

基于采样轨迹得到的累积奖励，是 $\mathbf{s}_t$ 下执行 $\mathbf{a}_t$ 后能获得的期望奖励”的采样估计，有偏差

$\hat{Q}_t^{(i)}$: estimate of expected reward if we take action $\mathbf{a}_t^{(i)}$ in state $\mathbf{s}_t^{(i)}$

can we get a better estimate?

真实期望累积奖励，它考虑了在当前策略下，从该状态-动作对出发，所有可能后续轨迹的奖励期望，是更准确的Q值定义

$Q(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_{\theta}} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: true *expected* reward-to-go

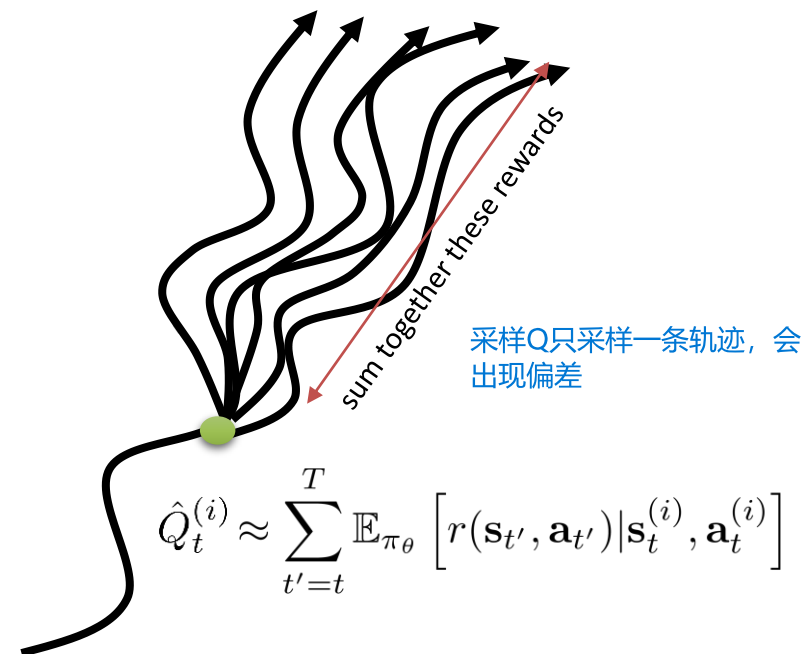
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) Q(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

真实的梯度近似，用真实Q替代采样Q

可行的优化方法：1. Q - learning

2. 神经网络。将状态和动作作为神经网络的输入，输出对应的Q值估计

但在实际场景中，真实Q往往是难以获取的



What about the baseline?

$Q(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: true *expected* reward-to-go

V: 状态价值函数, 即当前策略下, 处于状态 $\mathbf{s}_t$ 时能获得的期望累积奖励。

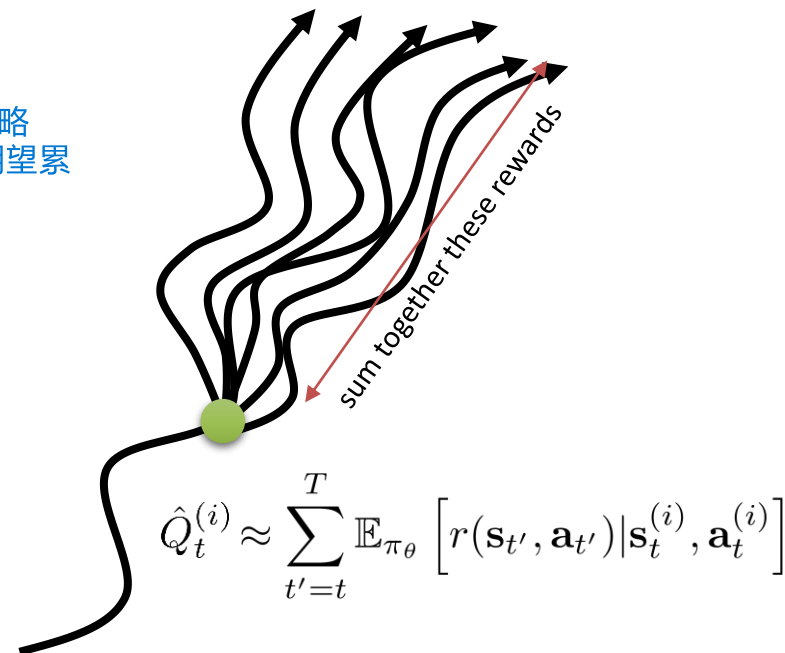
$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(Q(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - V(\mathbf{s}_t^{(i)}) \right)$$

$$A(\mathbf{s}_t, \mathbf{a}_t) = Q(\mathbf{s}_t, \mathbf{a}_t) - V(\mathbf{s}_t)$$

关键函数: advantage function, 核心作用是降低方差

b = average reward

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q(\mathbf{s}_t, \mathbf{a}_t)]$$



用优势函数后, 相当于只关注“这个动作比平均动作好多少”, 消除了状态本身带来的奖励差异, 让梯度更新更稳定。

State & state-action value functions

$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$

$V^\pi(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q^\pi(\mathbf{s}_t, \mathbf{a}_t)]$: total reward from $\mathbf{s}_t$

$A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$: how much better $\mathbf{a}_t$ is

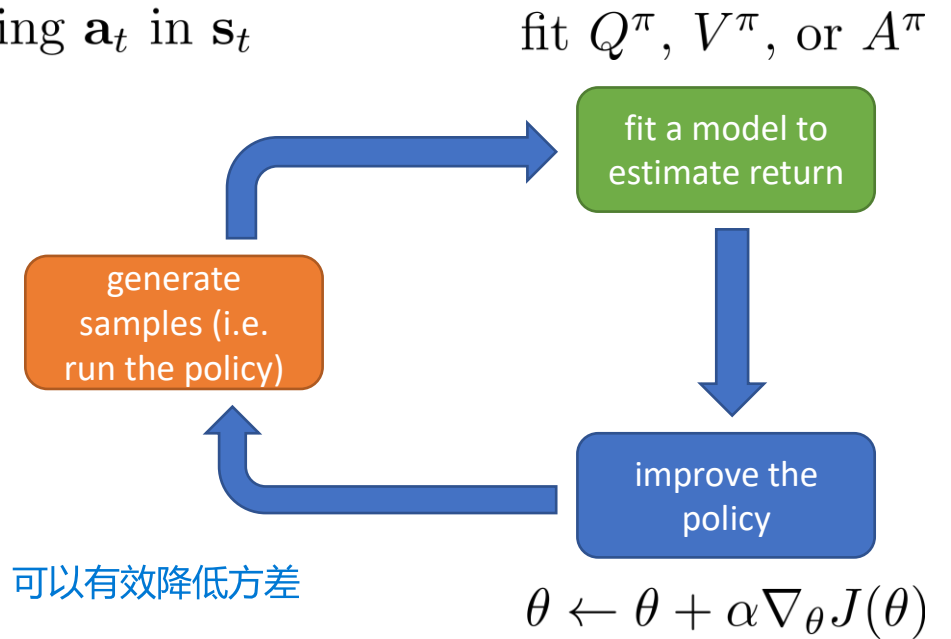
$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) A^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

注意减去的是状态价值的均值，可以有效降低方差

the better this estimate, the lower the variance

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\sum_{t'=1}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) - b_t \right)$$

unbiased, but high variance single-sample estimate



Part 2:

Policy evaluation

Value function fitting

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$$

$$V^\pi(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q^\pi(\mathbf{s}_t, \mathbf{a}_t)]$$

$$A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) A^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

fit *what* to *what*?

Q^π, V^π, A^π ?

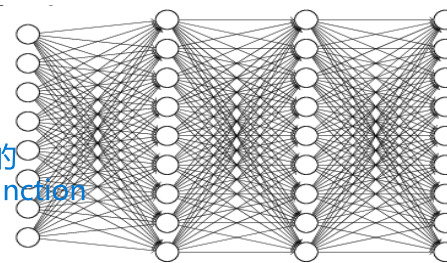
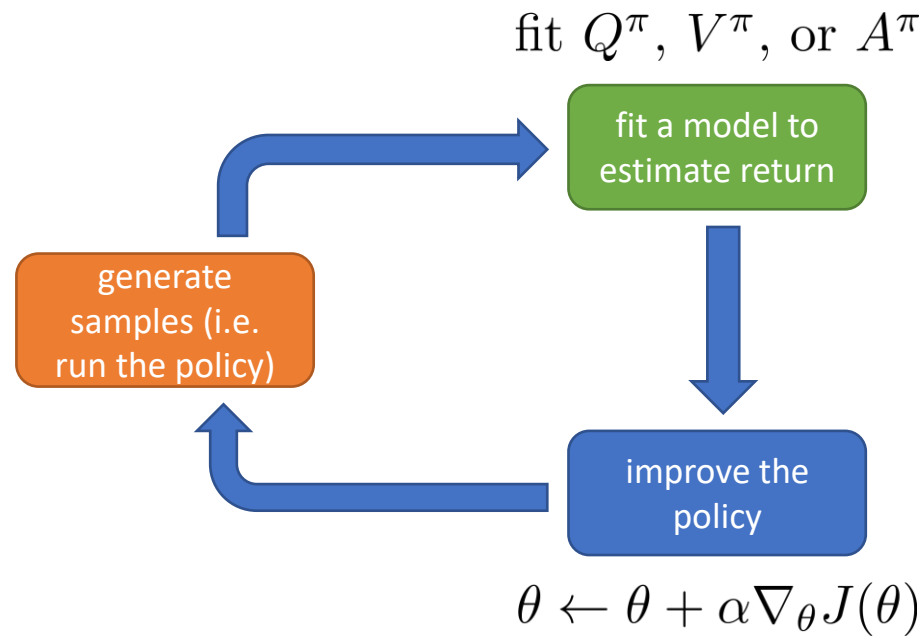
$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [V^\pi(\mathbf{s}_{t+1})]$$

$$A^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1}) - V^\pi(\mathbf{s}_t)$$

let's just fit $V^\pi(\mathbf{s})$!

由于Q和A function都可以通过V来推导

由于advantage function可以很好的预测最终的Q值，但是需要借助V function，所以我们只需要获得好的V估计即可



$\hat{V}^\pi(\mathbf{s})$

parameters ϕ

Policy evaluation

$$V^\pi(\mathbf{s}_t) = \sum_{t'=t}^H E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t]$$

$$J(\theta) = E_{\mathbf{s}_1 \sim p(\mathbf{s}_1)} [V^\pi(\mathbf{s}_1)]$$

how can we perform policy evaluation?

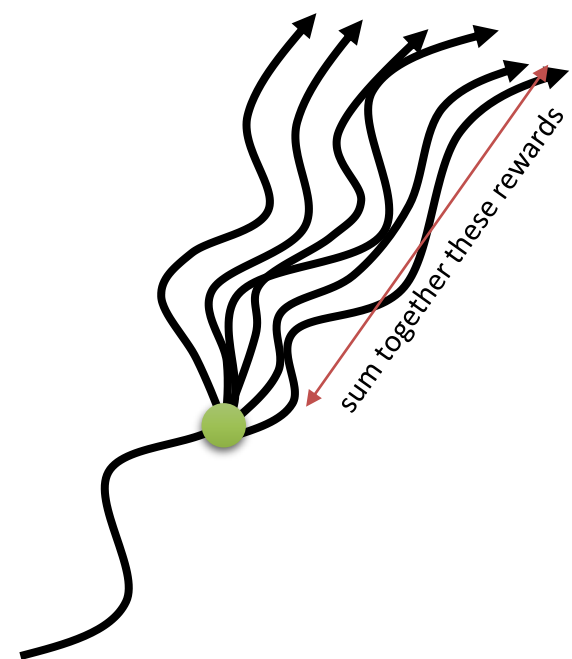
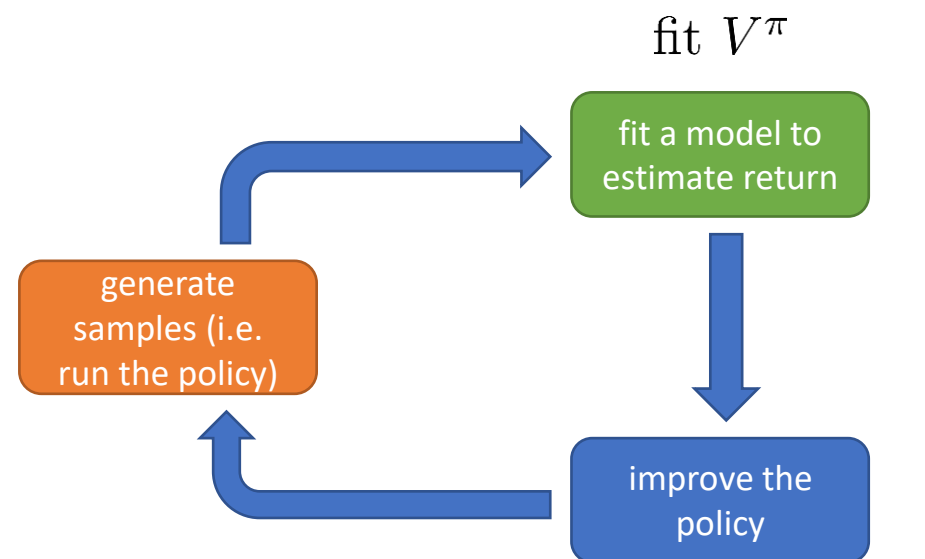
Monte Carlo policy evaluation (this is what policy gradient does)

蒙特卡洛策略评估：核心是通过采样来近似价值函数

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \text{单episode近似}$$

$$V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \text{多episode平均近似}$$

(requires us to reset the simulator)



Monte Carlo evaluation with function approximation

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$$

采样一次

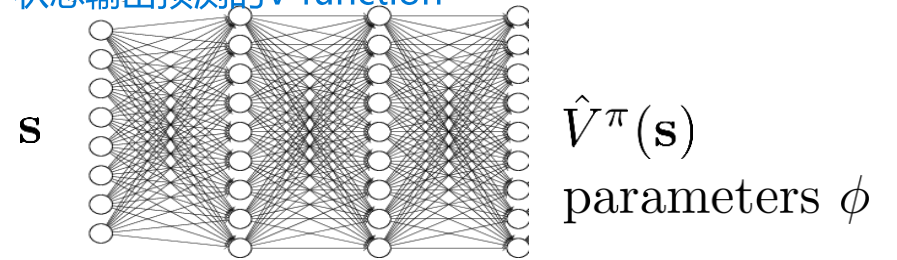
有监督的训练机器学习模型，输入某个状态输出预测的V function

not as good as this: $V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$

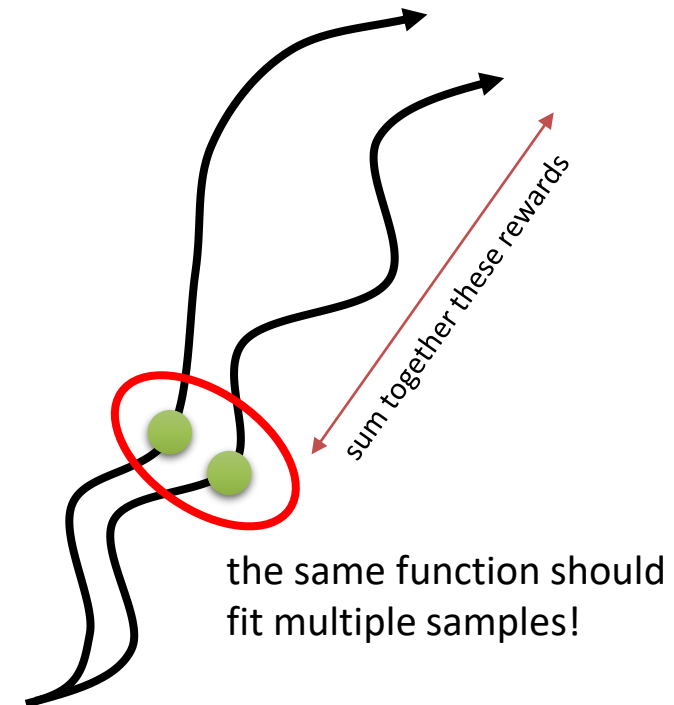
but still pretty good!

training data: $\left\{ \left(\mathbf{s}_t^{(i)}, \underbrace{\sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'})}_{y_t^{(i)}} \right) \right\}$

supervised regression: $\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - y_t^{(i)} \right\|^2$



采样N次



Can we do better?

基于自举 (Bootstrapped) 的蒙特卡洛策略评估优化

ideal target: $y_t^{(i)} = \sum_{t'=t}^H E_{\pi_\theta} \left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t^{(i)} \right] \approx r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + V^\pi(\mathbf{s}_{t+1}^{(i)}) \approx r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \underbrace{\hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})}$

Monte Carlo target: $y_t^{(i)} = \sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}^{(i)})$

training data: $\left\{ \underbrace{\left(\mathbf{s}_t^{(i)}, r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}) \right)}_{y_t^{(i)}} \right\}$

supervised regression: $\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - y_t^{(i)} \right\|^2$

sometimes referred to as a “bootstrapped” estimate

directly use previous fitted value function!

这是理论上最准确的状态价值目标，它基于贝尔曼方程，将当前状态的价值近似为即时奖励加上下一个状态的价值。该方法能利用已有模型信息，减少对完整episode采样的依赖。

贝尔曼方程的直观理解：

贝尔曼方程看作“当前状态的价值 = 立刻得到的奖励 + 后续状态价值的折现期望”

Discount factors

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - y_t^{(i)} \right\|^2$$

what if H (episode length) is ∞ ?

$\hat{V}_\phi^\pi$ can get infinitely large in many cases

假设每个现实世界的智能体会die

simple trick: better to get rewards sooner than later

γ changes the MDP:

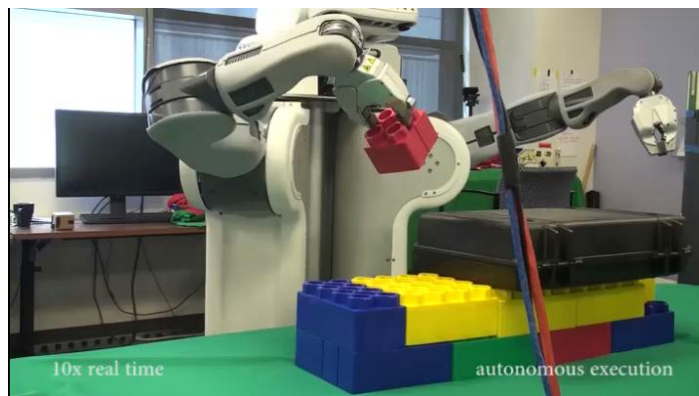
$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})$$

gamma代表每一步不死的概率，引入这一算子可以模拟每一步有1-gamma的概率死，这样智能体倾向于更早获得奖励

对下一状态的V function估计加gamma超参
这样的表示我们通常成为时序差分 (TD)

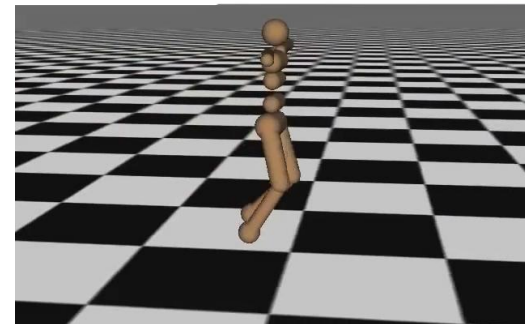
discount factor $\gamma \in [0, 1]$ (0.99 works well)

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

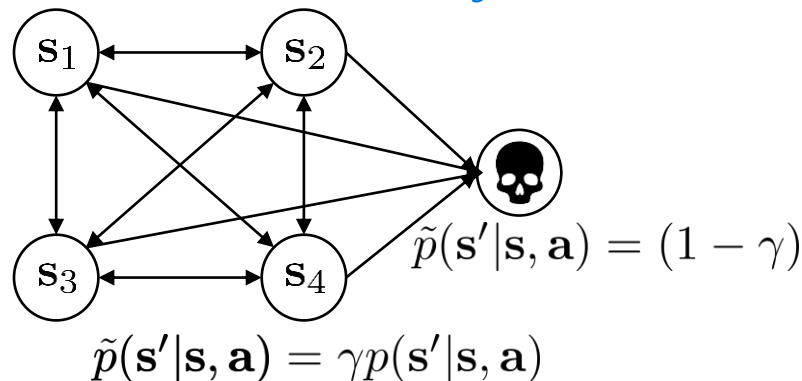


episodic tasks

Iteration 2000



continuous/cyclical tasks



Time varying 场景：比如股票交易策略训练，股票市场的行情、政策影响会随时间周期性波动

Time-varying vs. time-invariant

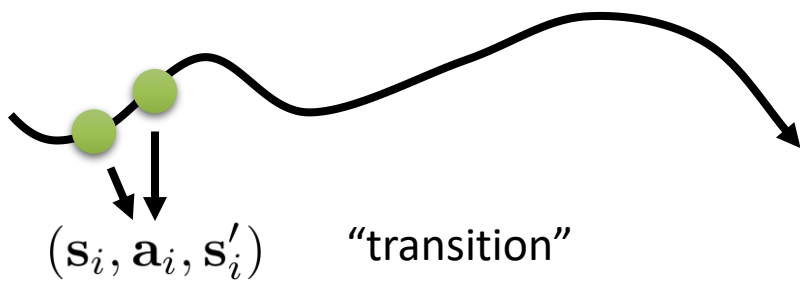
Time invariant : 经典的网格场景(Grid world), 网格的障碍, 奖励点都是固定的

training data: $\left\{ \left(\mathbf{s}_t^{(i)}, r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}) \right) \right\}$

$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \left\| \hat{V}_\phi^\pi(\mathbf{s}_t^{(i)}) - y_t^{(i)} \right\|^2$

we often don't actually care at which time step the data was collected

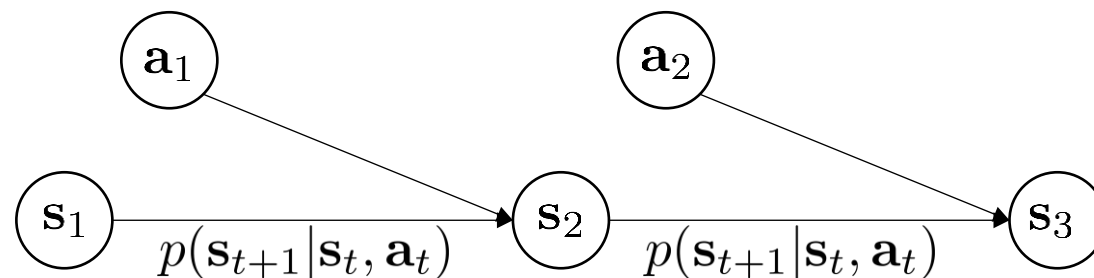
why?



In time invariant environment, we can ignore 't' and simplify the training data expression and loss function

training data: $\left\{ \left(\mathbf{s}_i, r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) \right) \right\}$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$



these are just notational conveniences,
we didn't actually change **anything**

Policy evaluation examples

TD-Gammon, Gerald Tesauro 1992

首次将时序差分 (TD) 学习与神经网络结合，用神经网络近似价值函数，解决了传统方法在复杂状态空间中的计算难题，是强化学习早期在棋牌类游戏里的里程碑应用

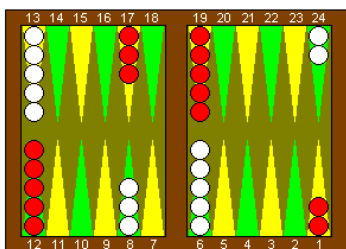


Figure 2. An illustration of the normal opening position in backgammon. TD-Gammon has sparked a near-universal conversion in the way experts play certain opening rolls. For example, with an opening roll of 4-1, most players have now switched from the traditional move of 13-9, 6-5, to TD-Gammon's preference, 13-9, 24-23. TD-Gammon's analysis is given in Table 2.

训练核心：通过大量对局数据，让模型学习在不同棋盘状态下，后续游戏的期望走向，以此指导智能体选择更优动作。

reward: game outcome

value function $\hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$:

expected outcome given board state

AlphaGo, Silver et al. 2016

训练核心：结合蒙特卡洛树搜索 (MCTS) 与价值函数，智能体在搜索过程中，通过价值函数快速评估潜在落子后的状态价值，大幅提升了搜索效率，最终实现了超越人类顶尖棋手的水平。



Figure 1. An illustration of the multilayer perception architecture used in TD-Gammon's neural network. This architecture is also used in the popular backpropagation learning procedure. Figure reproduced from [9].

reward: game outcome

value function $\hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$:

expected outcome given board state

技术方案：TD-Gammon侧重TD学习与神经网络的结合，AlphaGo则加入了蒙特卡洛树搜索，实现了更高效的搜索与决策。

Part 3:

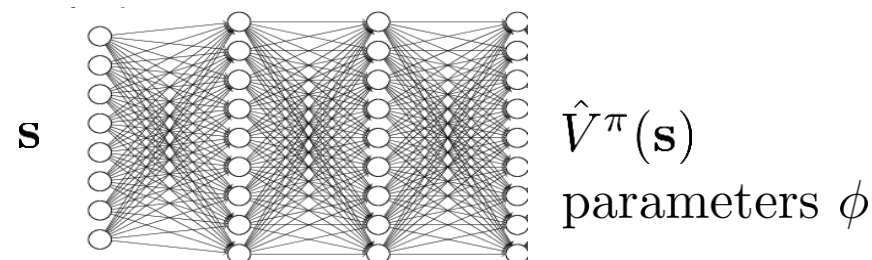
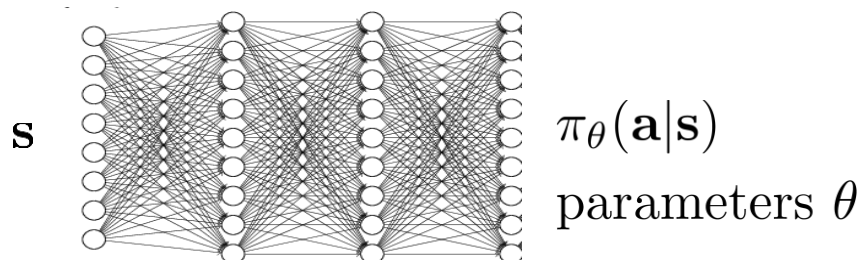
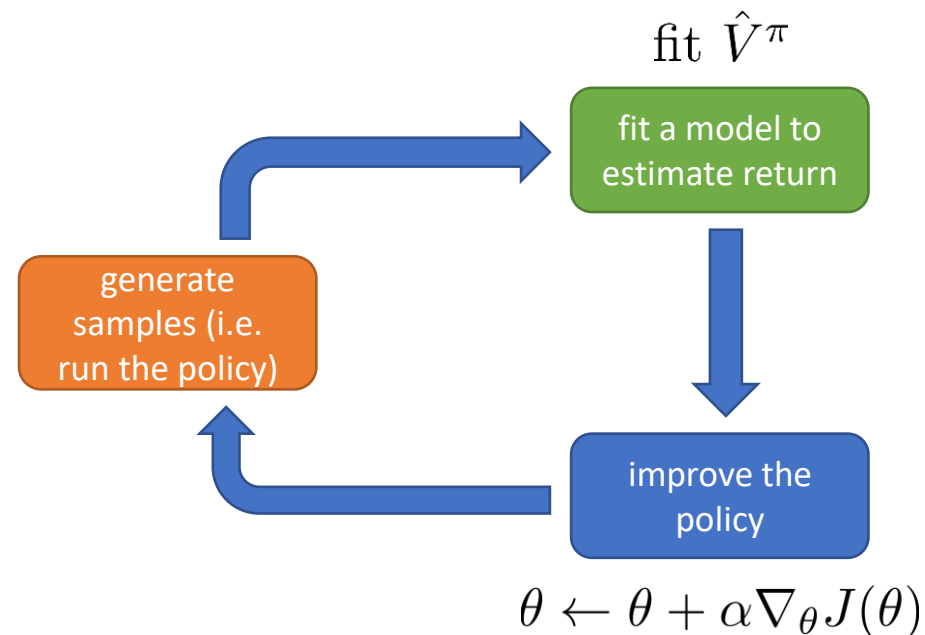
A basic actor-critic

A basic actor-critic algorithm

Actor-Critic算法结合了策略梯度方法和价值方法的优点：既通过Actor直接优化策略，又能通过Critic提供更稳定的梯度信号，相比单纯的策略梯度方法，收敛速度更快、稳定性更强，是强化学习中应用广泛的算法框架。

batch actor-critic algorithm:

1. sample $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run the policy)
2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$
3. refit $\hat{V}_\phi^\pi(\mathbf{s})$ to targets $\{y_i\}$
4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
5. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

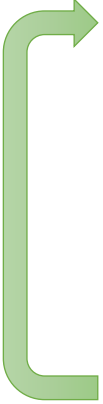


An online actor-critic algorithm

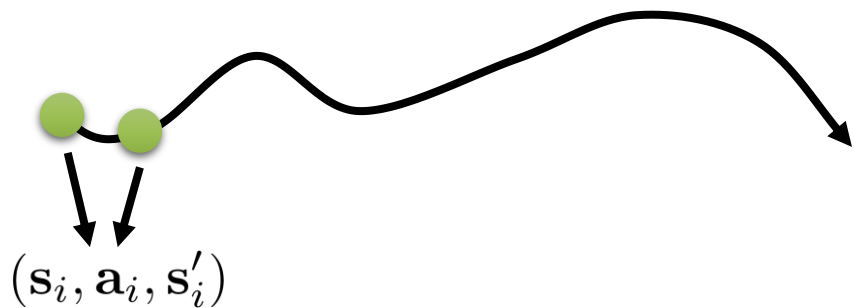
与basic版本的区别: basic版本采集批量样本再更新价值网络
而online版本是每进行一个版本, 一个步骤就更新一次网络来实现实时交互,
例如自动驾驶, 机器人在线控制, 在线广告推荐

batch actor-critic algorithm:

算法流程变为: 执行当前策略得到单个状态转移样本→立即更新价值网络→基于更新后的价值网络计算优势函数→更新策略参数, 实现边交互边学习的在线优化。

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run the policy)
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$
 3. refit $\hat{V}_\phi^\pi(\mathbf{s})$ to targets $\{y_i\}$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

what if we only have **one** transition?



$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$$

$$\mathcal{L}_i(\phi) = \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

$$\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_i(\phi)$$

$$\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$$

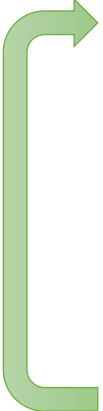
$$\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$$

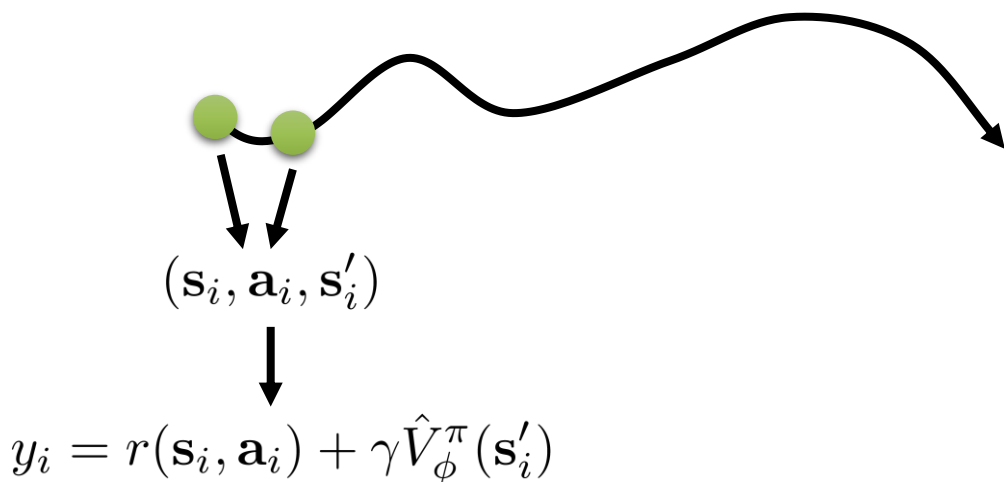
$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

An online actor-critic algorithm

A3C: A3C (Asynchronous Advantage Actor-Critic, 异步优势Actor-Critic), 它通过多个并行的智能体 (workers) 同时与环境交互, 每个智能体独立执行在线Actor-Critic的更新流程, 再将参数同步到全局模型, 解决了单智能体在线学习时样本效率低、收敛慢的问题。

online actor-critic algorithm:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$
 3. update ϕ using $\nabla_\phi \mathcal{L}_i(\phi)$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Example algorithm: A3C

Getting this to work in practice typically requires multiple parallel workers

无需存储批量样本, 内存占用更低, 适合资源受限的场景
能实时响应环境反馈, 实现边交互边学习, 适应动态变化的环境
保留了Actor-Critic框架的稳定性, 相比纯策略梯度方法收敛更快

$$\mathcal{L}_i(\phi) = \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

$$\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_i(\phi)$$

$$\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$$

$$\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$$

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Intermission

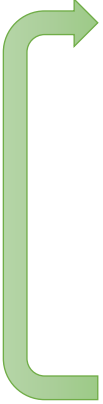
Part 4:

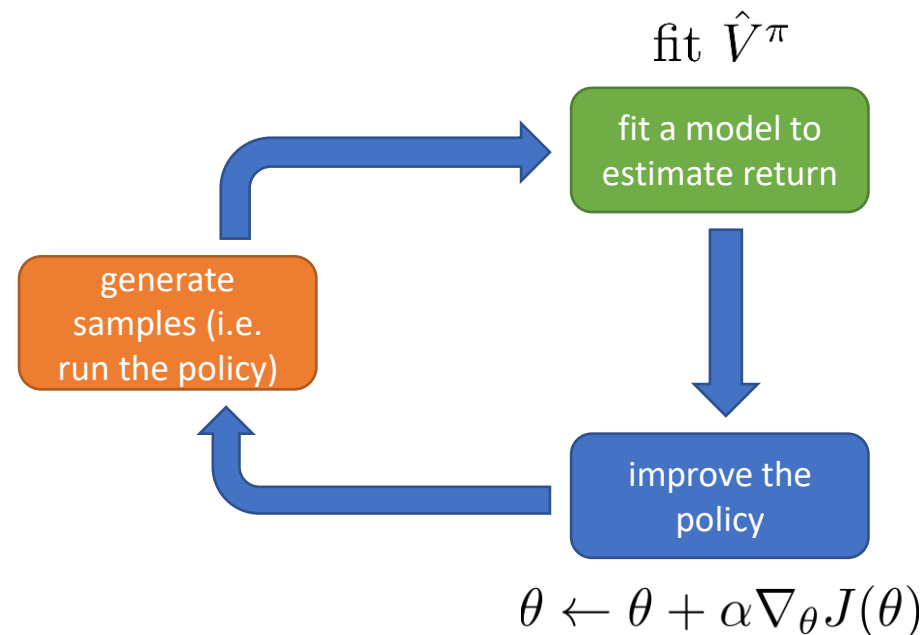
On-policy actor-critic

(a.k.a. policy gradient that actually works)

A basic actor-critic algorithm

batch actor-critic algorithm:

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run the policy)
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$
 3. refit $\hat{V}_\phi^\pi(\mathbf{s})$ to targets $\{y_i\}$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



What do we need to do to make this work in practice?

even better ways to estimate $\hat{A}^\pi$

better ways to estimate $\nabla_\theta J(\theta)$

e.g., GAE (today)

e.g., PPO (more on this later)

GAE (Generalized Advantage Estimation, 广义优势估计)。问题: 第 4 步中的单步 TD 估计虽然方差低, 但偏差 (Bias) 大。GAE 通过引入一个衰减因子, 在单步 TD 误差和多步 (甚至是全序列) 回报之间做平衡, 从而大幅提升训练的稳定性和效率。

PPO (Proximal Policy Optimization, 近端策略优化)。问题: 传统的梯度更新步长很难控制, 步子大了模型会崩溃。通过限制策略更新的幅度, 确保新旧策略之间不会偏差过大。

Critics and baselines

Actor-critic:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t^{(i)}) \right)$$

+ lower variance (due to critic)
- not unbiased (if the critic is not perfect)

Policy gradient:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\left(\sum_{t'=t}^H \gamma^{t'-t} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right) - b_t \right)$$

+ no bias
- higher variance (because single-sample estimate)

can we use $\hat{V}_{\phi}^{\pi}$ and still keep the estimator unbiased?

Exercise: prove that this is still unbiased

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\left(\sum_{t'=t}^H \gamma^{t'-t} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t^{(i)}) \right)$$

+ no bias

+ lower variance (baseline is closer to rewards)

Eligibility traces & n-step returns

$$\hat{A}_C^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}) - \hat{V}_\phi^\pi(\mathbf{s}_t)$$

+ lower variance

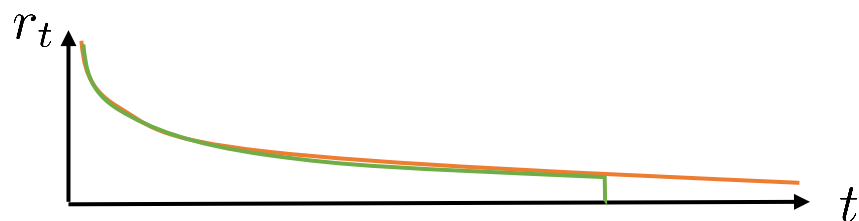
- higher bias if value is wrong (it always is)

$$\hat{A}_{MC}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t)$$

+ no bias

- higher variance (because single-sample estimate)

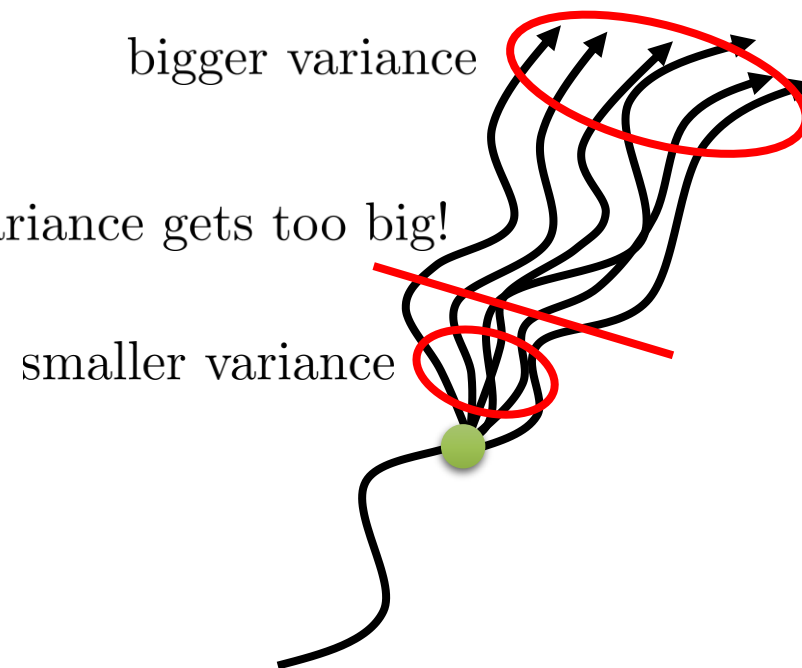
Can we combine these two, to control bias/variance tradeoff?



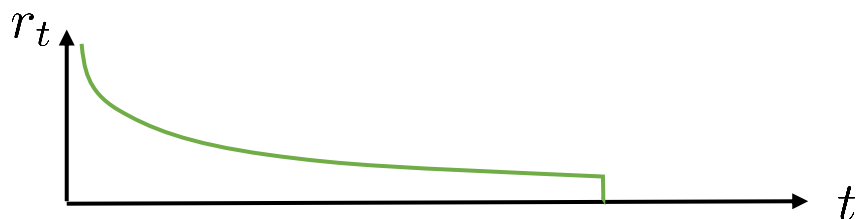
$$\hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n})$$

结合TD的低方差特性和MC的无偏性，在n步内使用真实回报，n步后用价值函数估计截断，平衡偏差和方差

choosing $n > 1$ often works better!



Generalized advantage estimation



Do we have to choose just one n ?

Cut everywhere all at once!

无需固定单一的 n ，而是将所有 n 步回报进行加权组合，相当于在回报序列的每个位置都进行截断，再通过权重调整不同截断方式的贡献，从而更精细地平衡偏差和方差。

$$\hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n})$$

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t)$$

weighted combination of n -step returns

How to weight?

Mostly prefer cutting earlier (less variance)

$$w_n \propto \lambda^{n-1}$$

exponential falloff

优先给更早的截断（即更小的 n ）分配更高权重，因为更早截断的回报估计方差更低。

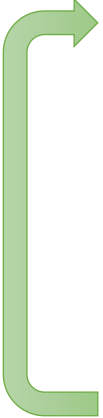
$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma((1 - \lambda)\hat{V}_\phi^\pi(\mathbf{s}_{t+1}) + \lambda(r(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) + \gamma((1 - \lambda)\hat{V}_\phi^\pi(\mathbf{s}_{t+2}) + \lambda r(\mathbf{s}_{t+2}, \mathbf{a}_{t+2}) + \dots))$$

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'} \quad \delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma\hat{V}_\phi^\pi(\mathbf{s}_{t'+1}) - \hat{V}_\phi^\pi(\mathbf{s}_{t'})$$

similar effect as discount!

Policy gradient with GAE

policy gradient with GAE:

- 
1. sample $\{\tau^{(i)}\}$ from π_θ (run the policy)
 2. evaluate $y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})$
 3. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to targets $\{y_t^{(i)}\}$
 4. evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}^{(i)}$
 5. $\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

can use GAE estimator here too!

Other good options:

use n -step return estimators

use Monte Carlo estimator with $\hat{V}^\pi(\mathbf{s}_t)$ baseline

in practice, we typically
center the advantages

$$\mu = \frac{1}{HN} \sum_{i,t} \hat{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

$$\sigma = \sqrt{\frac{1}{HN} \sum_{i,t} \left(\hat{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - \mu \right)^2}$$

$$\bar{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \frac{\hat{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - \mu}{\sigma}$$

advantage function归一化

This is the first part of implementing PPO

The other part is a slightly different gradient estimator

这页PPT的内容是PPO算法实现的第一部分，PPO的另一部分是稍有不同的梯度估计器，通过结合GAE的优势估计和特殊的梯度更新方式，PPO在保证策略更新稳定性的同时，实现了高效的策略优化。

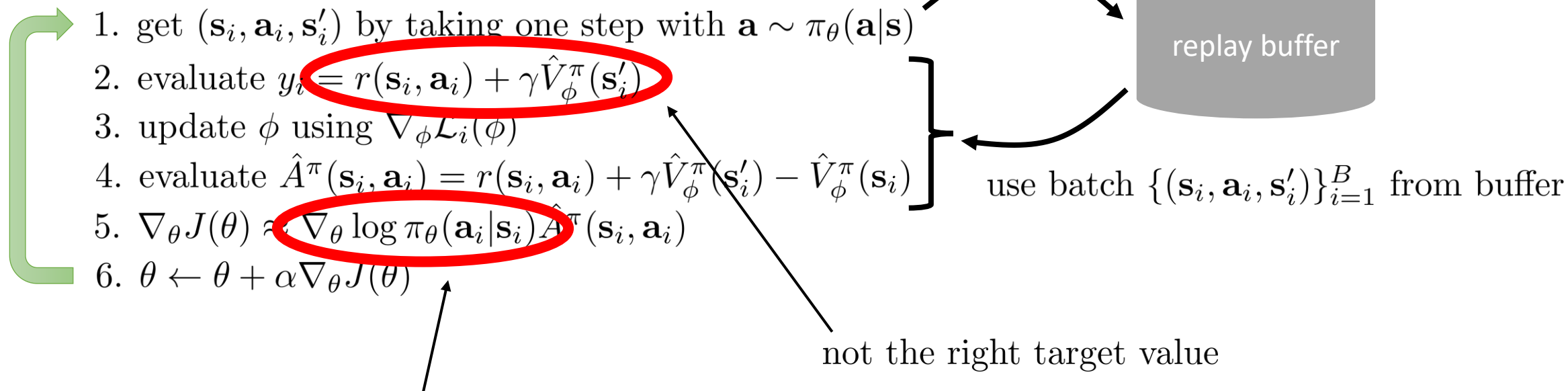
Part 5:

Off-policy actor-critic

(a.k.a. actor-critic)

Actor-critic with a replay buffer

online actor-critic algorithm:



not the action π_θ would have taken!

This algorithm is broken!

Can you spot the problems?

Hint: $(\mathbf{s}_i, \mathbf{a}_i)$ no longer comes from latest $\pi_\theta(\mathbf{a}_i|\mathbf{s}_i)$

因为回放缓冲区中的数据是之前策略生成的，当策略更新后，再使用旧数据训练可能导致训练不稳定或不准确。

Key idea: estimate Q-function

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$$

where $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ came from an older policy $\bar{\pi}$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

why is this wrong?

↑

$$\text{want } V^\pi(\mathbf{s}_i) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [r(\mathbf{s}_i, \mathbf{a}) + \gamma V^\pi(\mathbf{s}')]]$$

$$\text{got } V^\pi(\mathbf{s}_i) = \mathbb{E}_{\mathbf{a} \sim \bar{\pi}(\mathbf{a}|\mathbf{s}_i)} [r(\mathbf{s}_i, \mathbf{a}) + \gamma V^\pi(\mathbf{s}')]]$$

$$\text{because } \mathbf{a}_i \sim \bar{\pi}(\mathbf{a}|\mathbf{s}_i)$$

What if we estimate Q-function instead?

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \left\| \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i \right\|^2$$

to estimate:

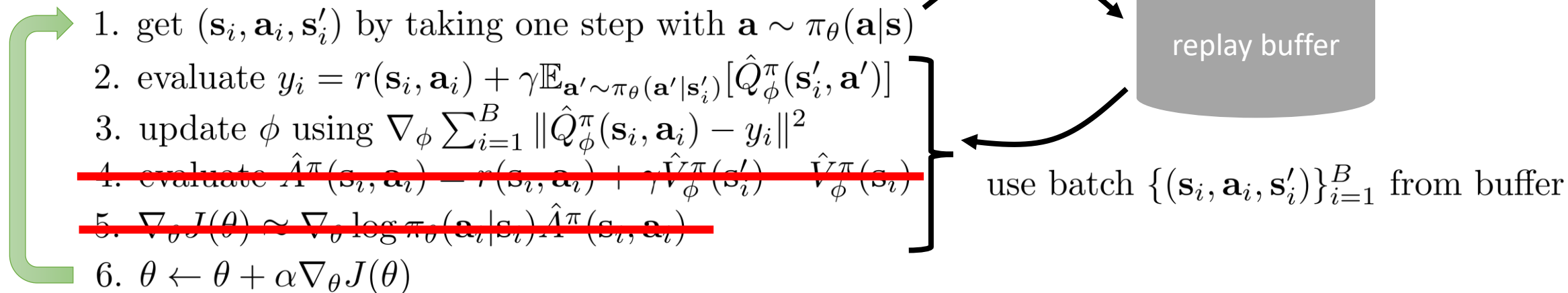
$$\mathbf{a}'_i \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)$$

Note that we use the **latest** policy!

$$\mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]] \approx \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}'_i)$$

Actor-critic with a replay buffer

online actor-critic algorithm:



回放缓冲区用于存储历史的状态-动作-下一状态数据，训练时从中采样一批数据进行批量训练，这种方式可以打破数据间的时序相关性，提升数据利用效率，让训练更稳定。

use the same trick, but this time for $\mathbf{a}_i$ rather than $\mathbf{a}'_i$!

sample $\mathbf{a}_i^\pi \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)$

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi | \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi)$$

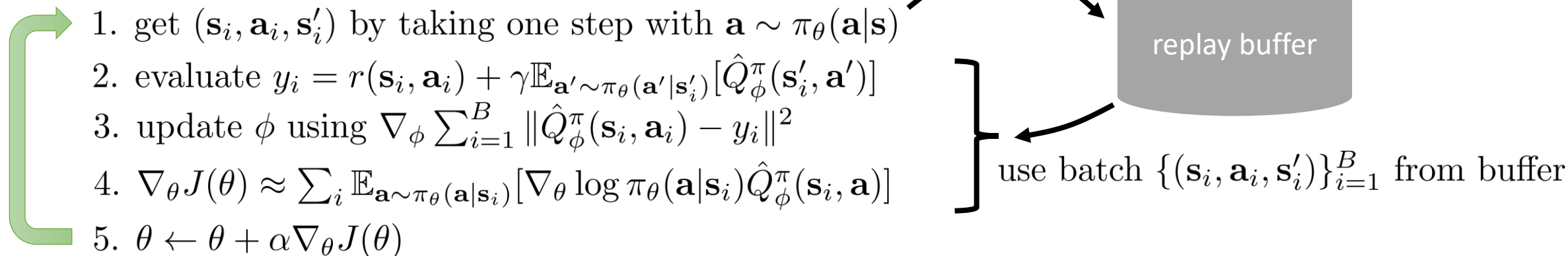
↑
not from replay buffer $\mathcal{R}$!

higher variance, but convenient
why is higher variance OK here?

in practice: $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi | \mathbf{s}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi)$

Actor-critic with a replay buffer

off-policy actor-critic algorithm:



Is there any remaining problem?

$\mathbf{s}_i$ didn't come from $p_\theta(\mathbf{s})$

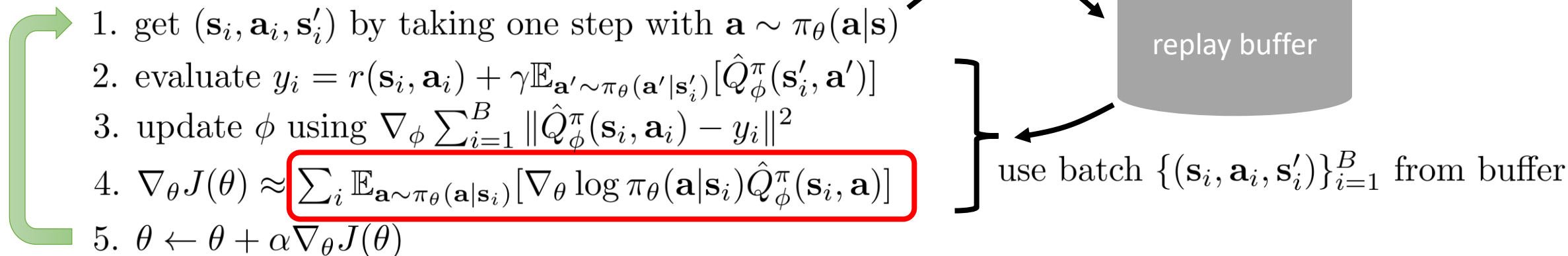
nothing we can do here, just accept it

intuition: we want optimal policy on $p_\theta(\mathbf{s})$

but we get optimal policy on a *broader* distribution

An implementation trick

off-policy actor-critic algorithm:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)}[\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
 3. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 4. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)}[\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

which gradient is this estimating?

$$\nabla_\theta \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)}[\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$$

this is a neural network!

we can compute $\frac{d}{d\mathbf{a}} \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})$

can we use this?

Reparameterization trick

say $\pi_\theta(\mathbf{a}|\mathbf{s}) = \mathcal{N}(\mu_\theta(\mathbf{s}), \sigma_\theta(\mathbf{s}))$

how do we sample?

$$\epsilon \sim \mathcal{N}(0, 1)$$

$$\mathbf{a} = \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon$$

$$\nabla_\theta \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \approx \nabla_\theta Q(\mathbf{s}, \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon)$$

“reparameterized gradient”


$$\mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, 1)} [Q(\mathbf{s}, \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon)]$$

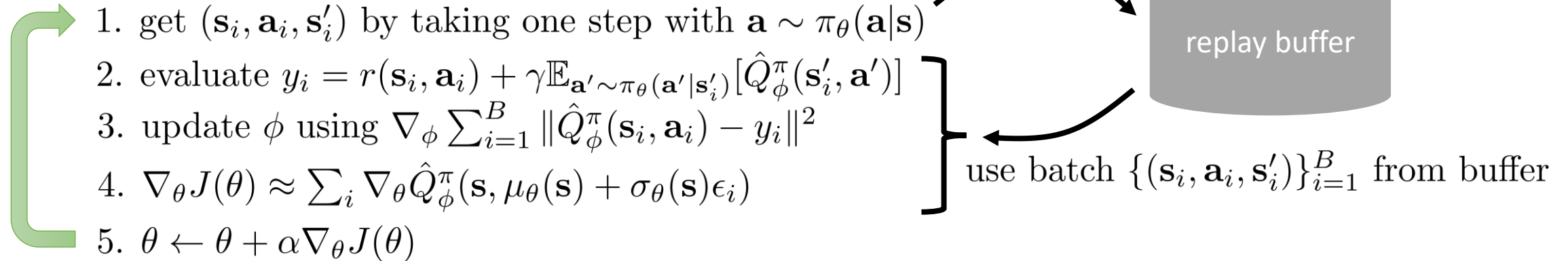
$$\approx Q(\mathbf{s}, \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon) \quad \epsilon \sim \mathcal{N}(0, 1)$$



this is easy to differentiate with any
autograd library (e.g., PyTorch)

Off-policy actor-critic with reparameterization

off-policy reparameterized actor-critic algorithm:



Example practical algorithm:

Soft actor-critic

- uses a few more tricks for Q-function estimation (we'll see this in Q-learning lectures)
- uses entropy regularization (we'll learn about this later)

We'll also learn about algorithms that do this with deterministic policies later!